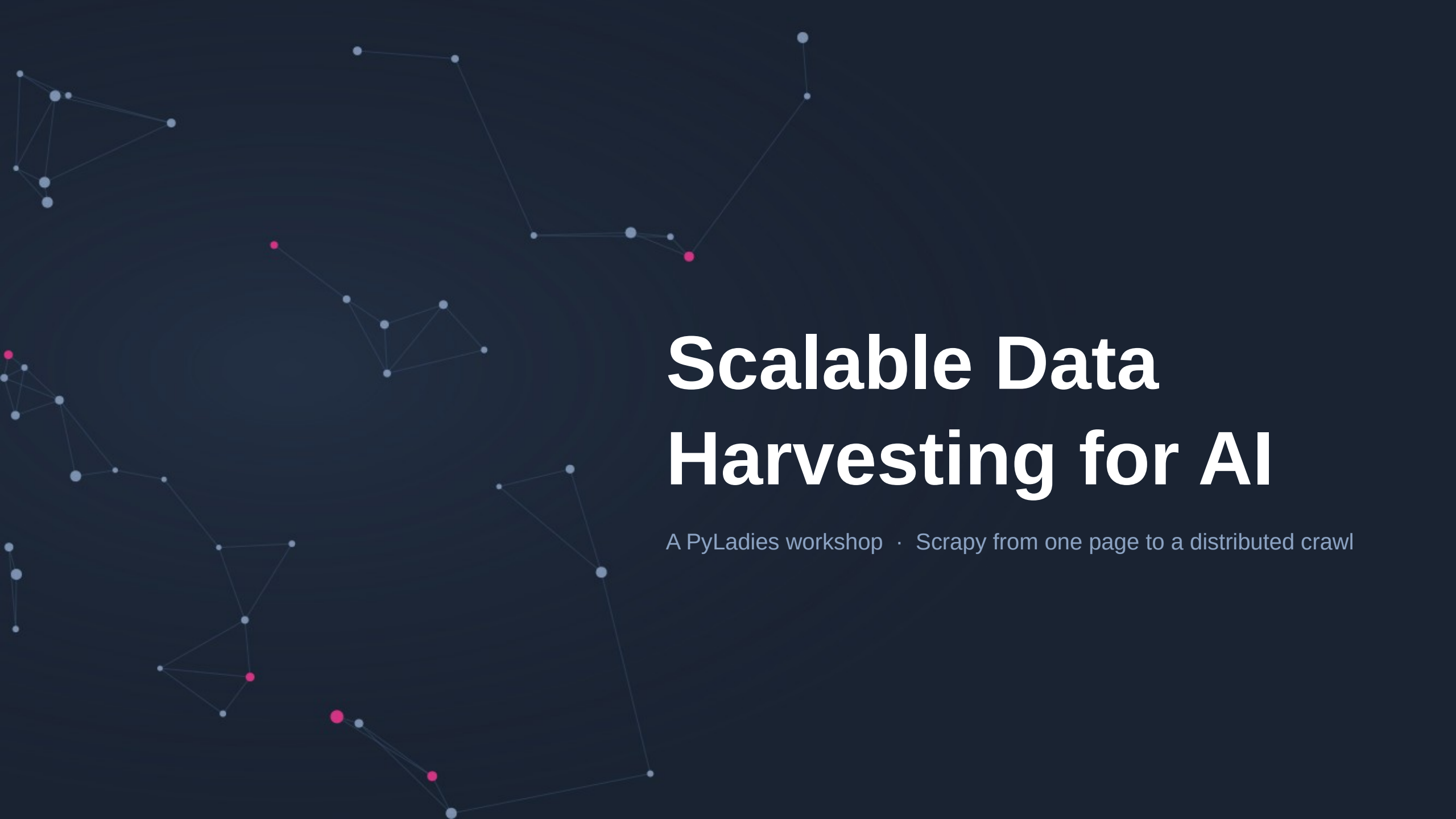


An abstract network diagram with various clusters of nodes connected by lines. Some nodes are highlighted in red, while others are light blue. The background is a dark navy blue.

Scalable Data Harvesting for AI

A PyLadies workshop · Scrapy from one page to a distributed crawl

Your speakers tonight



- Data Engineer and Tech Lead at Objective Platform, passionate about making technical knowledge accessible.
- Long-time PyLadies member who has organised and led multiple workshops, here to guide you through Scrapy tonight.



- Background in AI text classification research and AI consulting, helping clients adopt AI tools.
- Now evolving into data engineering, focused on how data is harvested for machine learning pipelines.

Workshop overview

EX 0 Setting up a Scrapy project and your first spider

EX 1 Selectors and extracting data from a single page

EX 2 Pagination and following links across pages

EX 3a Parsing JavaScript with a headless browser

EX 3b Distributed scraping with Redis

What is web scraping?

WHY IS IT NECESSARY?

- The web is the largest source of training and fine-tuning data
- Most useful data has no public API, so scraping is the only way in

HOW IT FEEDS AI PIPELINES

- Builds training corpora, RAG knowledge bases and evaluation sets
- Keeps models fresh by refreshing data on a schedule

SOME OF THE LIBRARIES AND FRAMEWORKS YOU'LL SEE



Scrapling



Playwright

What is Scrapy?

THE DEFINITION

- An open-source Python framework built for web scraping at scale
- Handles requests, parsing and data pipelines in one place

WHY USE IT

- Asynchronous by default, so it scrapes hundreds of pages in parallel
- Built-in retries, throttling and duplicate filtering save you boilerplate
- Scales from one spider to a distributed crawl across many machines



Beautiful Soup vs Scrapy

Beautiful Soup

A PARSING LIBRARY

A sharp tool. Quick scripts and one-off jobs.

- Parses HTML you've already fetched
- Synchronous, one page at a time
- You handle retries and throttling

Scrapy

A FULL FRAMEWORK

An entire factory. Production pipelines at scale.

- Fetches, parses and stores in one tool
- Asynchronous, many pages in parallel
- Retries and throttling built in

EXERCISE 0

Set up your project and first spider

THE GOAL

- Start a new Scrapy project
- Generate an empty spider for books.toscrape.com
- Run it and confirm a 200 response

KEY COMMANDS

```
scrapy startproject <name>
```

Create a new Scrapy project

```
scrapy genspider <name> <domain>
```

Generate an empty spider

```
scrapy crawl <name>
```

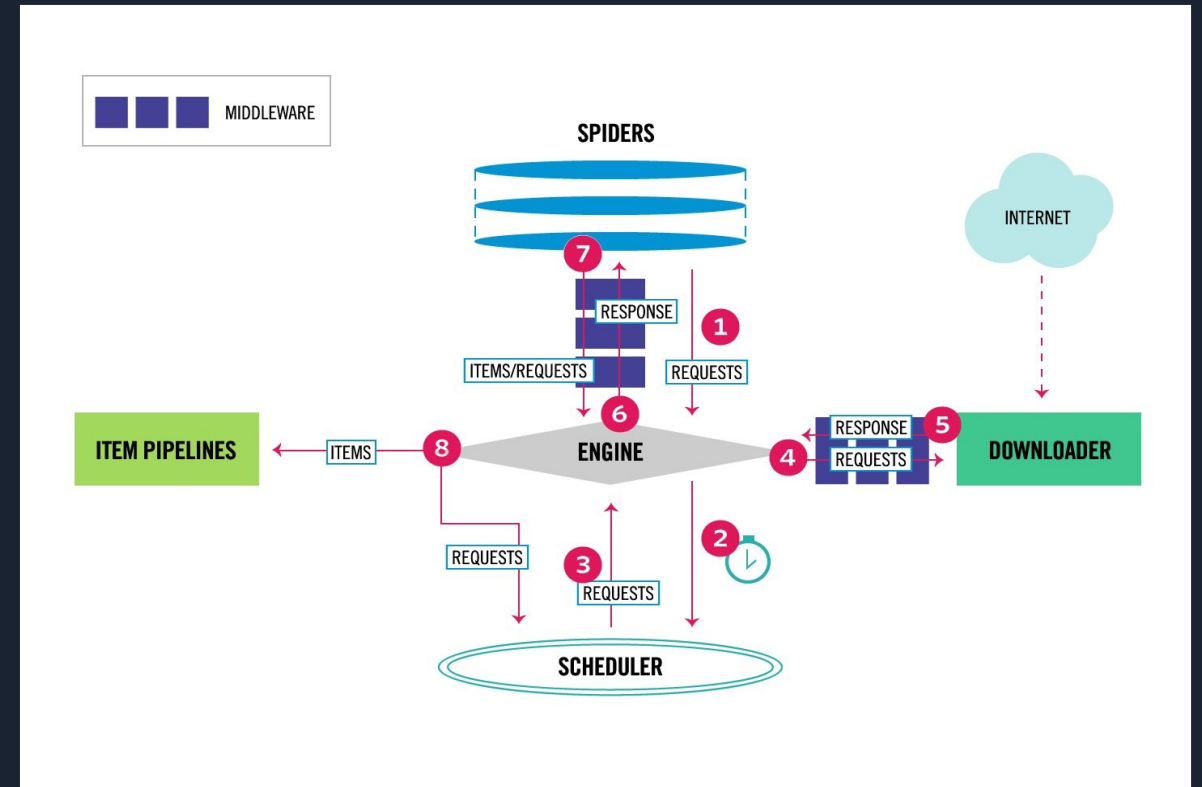
Run it and check the response

How Scrapy works under the hood

THE BIG PICTURE

- Five moving parts that talk through a central engine
- Spiders define what to scrape, the engine routes everything
- Pipelines clean and store the items at the end

Source: [official Scrapy documentation](#)



EXERCISE 1

Scraping individual pages

THE GOAL

- Add one book URL to start_urls
- Define your schema in items.py
- Write CSS selectors in the parse method
- Use a pipeline to clean and persist data
- Output a CSV with one row of data

KEY COMMANDS

```
scrapy shell <url>
```

Open Scrapy's interactive shell

```
view(response)
```

See the response as Scrapy sees it

```
response.css("h1::text").get()
```

Extract text with a fallback

```
response.css("img::attr(src)").get()
```

Extract an attribute value

```
scrapy crawl <name> -o output.csv
```

Run the spider and write to CSV

Spider to scheduler

1

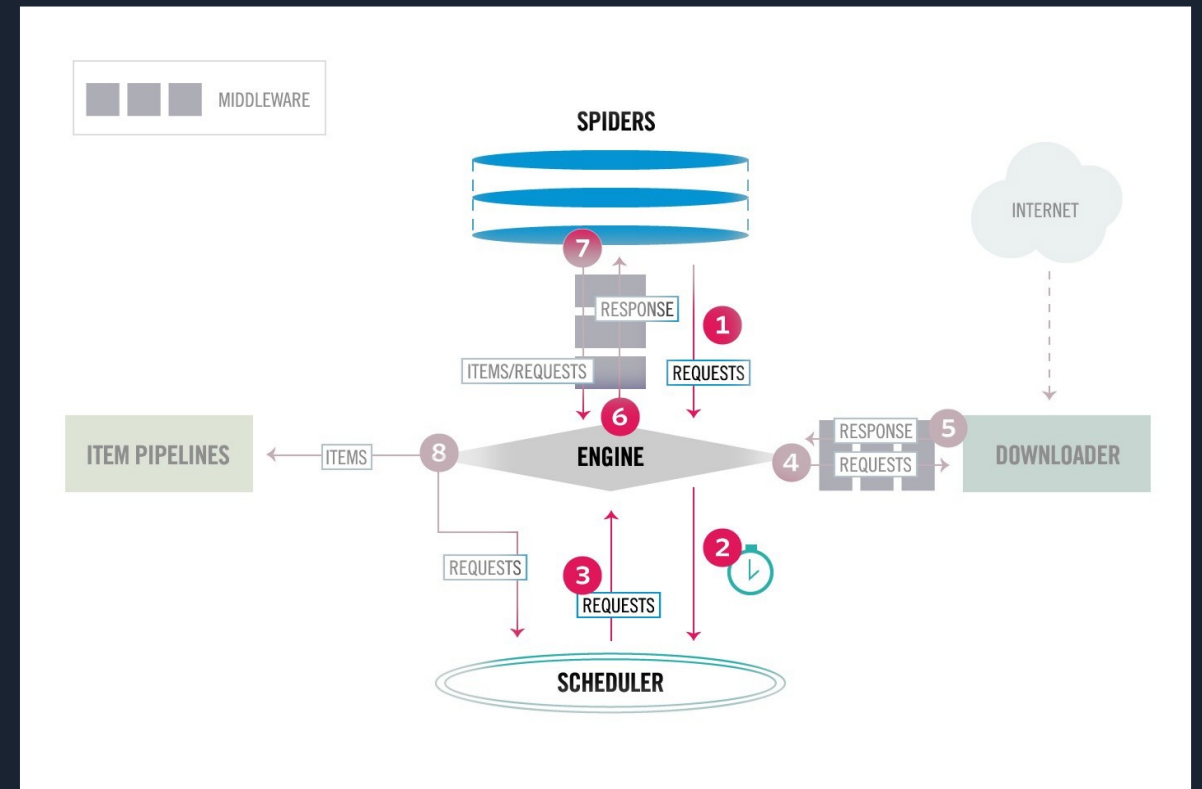
The spider sends its first requests to the engine

2

The engine hands the requests to the scheduler, which queues them

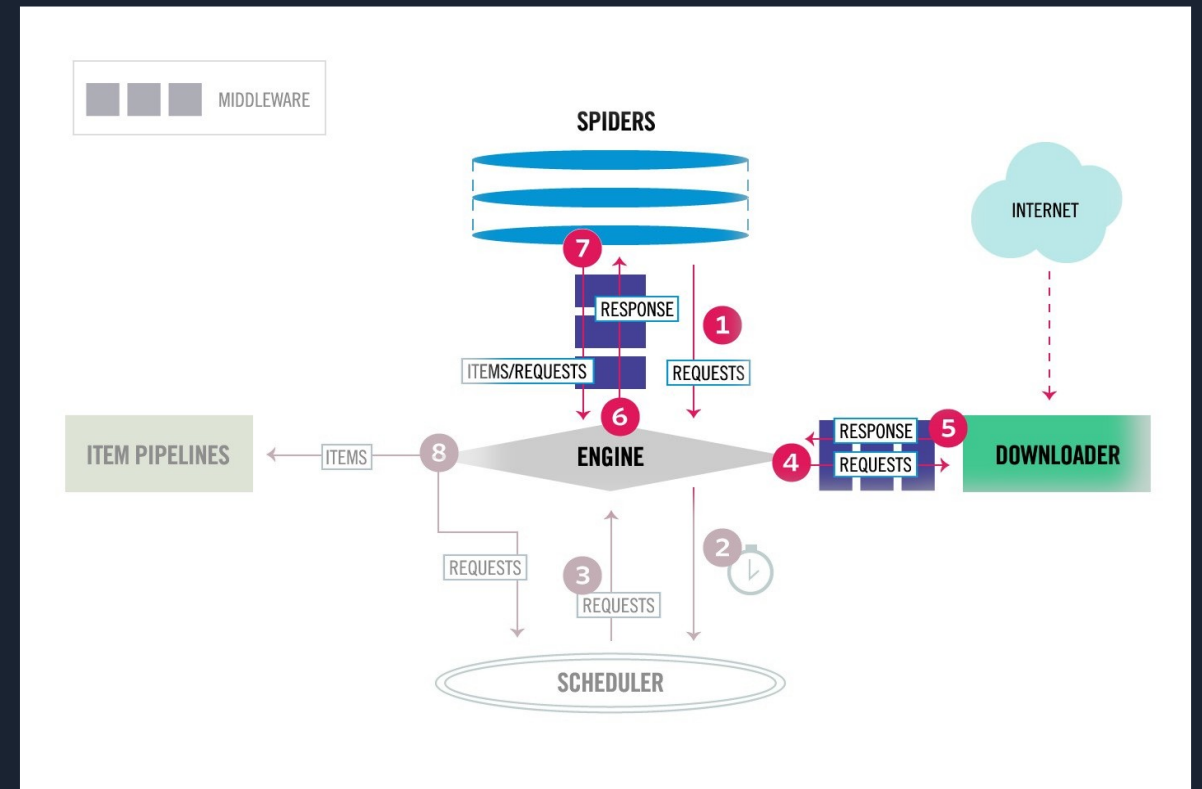
3

The scheduler hands the next request back to the engine when it's ready



Downloader does the heavy lifting

- 4 The engine sends the request to the downloader through middleware
- 5 The downloader fetches the page from the internet and returns a response
- 6 The engine passes the response back to the spider for parsing

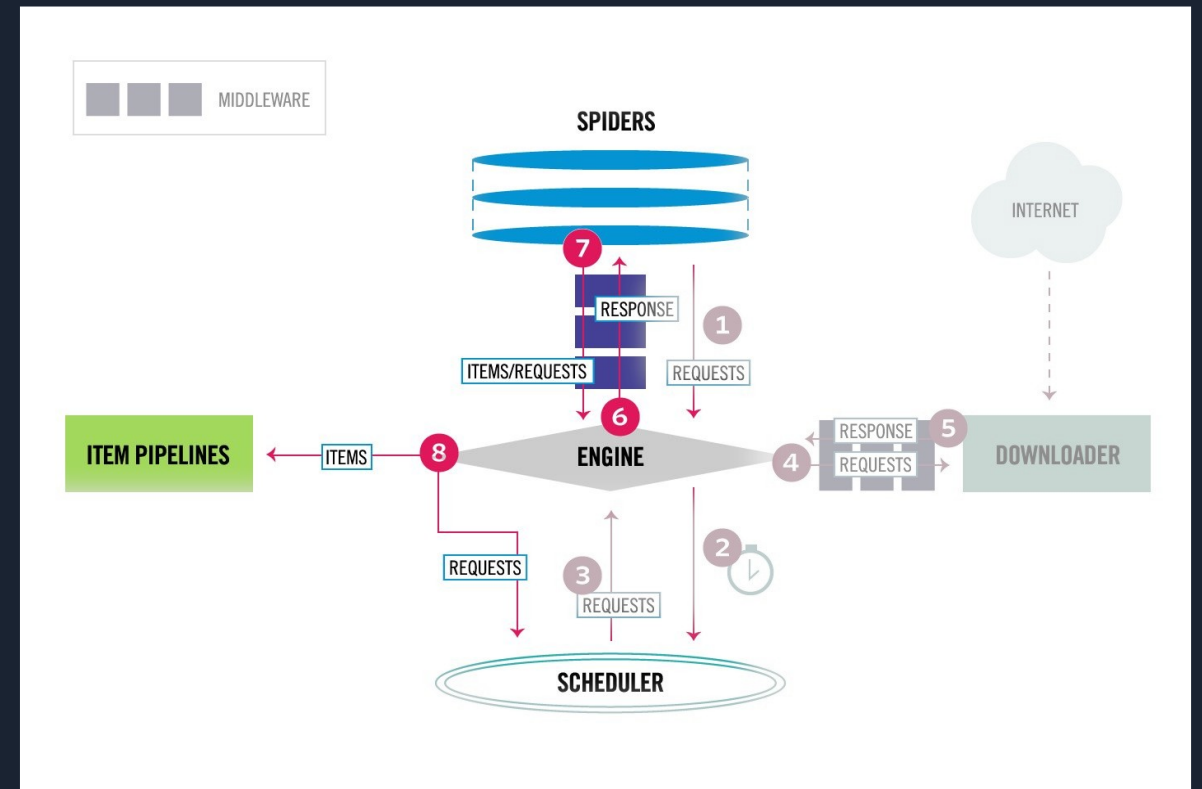


Items flow into pipelines

7 The spider parses the response into items, and may yield new requests to follow

8 The engine sends items to the pipelines for cleaning and storage, and queues any new requests

The loop repeats until the scheduler's queue is empty.



EXERCISE 2

Pagination and following links

THE GOAL

- Point `start_urls` at the listing page
- Extract book links and the next-page link (or derive page URLs)
- Split logic: `parse` for links, `parse_book` for items
- End with a CSV of 1,000 rows

KEY COMMANDS

```
response.follow(url, callback=self.parse_book)
```

Follow a link to another method

```
scrapy crawl <name> -L INFO -O output.csv
```

Run with cleaner logs

```
CONCURRENT_REQUESTS_PER_DOMAIN = 64
```

Run many requests in parallel

```
DOWNLOAD_DELAY = 0
```

Remove the wait (use carefully)

```
AUTOTHROTTLING_ENABLED = True
```

Let Scrapy adjust speed

Scraping JavaScript pages with a headless browser

WHY YOU NEED ONE

- Many sites load content with JavaScript after the page arrives
- A normal request only sees the unrendered HTML, missing the data
- A headless browser runs the JavaScript and gives you the full page

TOOLS YOU'LL SEE

Playwright

Modern browser automation, controls real Chromium

scrapy-playwright

Plugs Playwright into Scrapy as a downloader

Scrapling

Adaptive scraper with browser support built in

Looking human and getting past logins

AVOIDING BANS

- Randomise cursor movements and scroll patterns
- Rotate user agents and add realistic delays
- Scrapling has built-in stealth mode for this

GETTING PAST LOGINS

- Enable cookies to stay logged in across requests
- Add your auth token to request headers manually
- Click buttons with the headless browser when needed

Scrapling — [GitHub repo](#) · [Documentation](#)

Always check the site's terms of service before scraping behind a login

EXERCISE 3b

Distributed scraping with Redis

THE GOAL

- Pagination spider pushes URLs to a Redis queue
- Worker spiders pop URLs and scrape in parallel
- Scheduler and dupe filter both run on Redis
- Each worker writes to its own output file

KEY PIECES

```
uv add scrapy-redis
```

Adds Redis support to your project

```
docker compose up --service redis
```

Start the Redis instance

```
class Worker(scrapy_redis.RedisSpider)
```

Pull URLs from the queue

```
redis.lpush(queue_key, *urls)
```

Push absolute URLs onto the queue

Full setup, settings and run commands in the README

Thank you!

Questions?

Workshop materials and solutions:

github.com/pyladiesams/scalable-data-harvesting-for-ai-may2026